

Potential Technical Debt and Its Resolution in Code Reviews: An Exploratory Study of the OpenStack and Qt Communities

Liming Fu^{1,2}, Peng Liang^{1,2*}, Zeeshan Rasheed^{1,2}, Zengyang Li³, Amjed Tahir⁴, Xiaofeng Han^{1,2}

¹School of Computer Science, Wuhan University, Wuhan, China

²Hubei LuoJia Laboratory, Wuhan, China

³School of Computer Science & Hubei Provincial Key Laboratory of Artificial Intelligence and Smart Learning, Central China Normal University, Wuhan, China

⁴School of Mathematical and Computational Sciences, Massey University, Palmerston North, New Zealand

{limingfu, liangp, zeeshanrasheed, hanxiaofeng}@whu.edu.cn, zengyangli@ccnu.edu.cn, a.tahir@massey.ac.nz

ABSTRACT

Background: Technical Debt (TD) refers to the situation where developers make trade-offs to achieve short-term goals at the expense of long-term code quality, which can have a negative impact on the quality of software systems. In the context of code review, such sub-optimal implementations have chances to be timely resolved during the review process before the code is merged. Therefore, we could consider them as Potential Technical Debt (PTD) since PTD will evolve into TD when it is injected into software systems without being resolved. **Aim:** To date, little is known about the extent to which PTD is identified in code reviews. Many tools have been provided to detect TD, but these tools lack consensus and a large amount of PTD are undetectable by tools while code review could help verify the quality of code that has been committed by identifying issues, such as PTD. To this end, we conducted an exploratory study in an attempt to understand the nature of PTD in code reviews and track down the resolution of PTD after being identified. **Method:** We randomly collected 2,030 review comments from the Nova project of OpenStack and the Qt Base project of Qt. We then manually checked these review comments, and obtained 163 PTD-related review comments for further analysis. **Results:** Our results show that: (1) PTD can be identified in code reviews but is not prevalent. (2) Design, defect, documentation, requirement, test, and code PTD are identified in code reviews, in which code and documentation PTD are the dominant. (3) 81.0% of the PTD identified in code reviews has been resolved by developers, and 78.0% of the resolved TD was resolved by developers within a week. (4) Code refactoring is the main practice used by developers to resolve the PTD identified in code reviews. **Conclusions:** Our findings indicate that: (1) review-based detection of PTD is seen as one of the trustworthy mechanisms in development, and (2) there is still a significant proportion of PTD (19.0%) remaining unresolved when injected into the software systems. Practitioners and researchers should establish effective strategies to manage and resolve PTD in development.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEM '22, September 19–23, 2022, Helsinki, Finland

© 2022 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145.nnnnnnnnnnn>

CCS CONCEPTS

• **Software and its engineering** → **Designing software**; • **General and reference** → **Empirical studies**.

KEYWORDS

Technical Debt, Potential Technical Debt, Code Review, OpenStack

ACM Reference Format:

Liming Fu^{1,2}, Peng Liang^{1,2*}, Zeeshan Rasheed^{1,2}, Zengyang Li³, Amjed Tahir⁴, Xiaofeng Han^{1,2}. 2022. Potential Technical Debt and Its Resolution in Code Reviews: An Exploratory Study of the OpenStack and Qt Communities. In *16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '22)*, September 19–23, 2022, Helsinki, Finland. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145.nnnnnnnnnnn>

1 INTRODUCTION

Developers sometimes make compromised decisions between a shorter completion time and better software quality, and might take shortcuts to meet short-term goals, such as the need for releasing a new feature or fixing a bug. These technical compromises can yield quick and short-term benefits, but may also hurt the long-term health of a software system [22]. In this context, the Technical Debt (TD) metaphor was framed by Cunningham [4], which refers to the unavoidable maintenance and evolution costs of such not-quite-right solutions in development. Prior studies have shown that TD can have a negative impact on the quality of software systems [37].

Over the past two decades, many studies have focused on TD and contributed to the community about the nature, detection, and repayment of TD, especially Self-Admitted Technical Debt (SATD), which refers to the situation where developers know that the current implementation is not optimal and write comments alerting the inadequacy of the solution [25]. However, to the best of our knowledge, little is known about the presence of TD in code reviews. Code review is the process of manually reviewing and verifying the code that has been committed by developers. It can verify a wide range of issues in the code from potential bugs and security violations to code quality issues (e.g., code smells). Those issues can also include TD because developers may intentionally or unintentionally write code that can be considered as sub-optimal during the development process [7]. Fortunately, after code reviewers detect the issues, such sub-optimal implementations have the chances to be timely resolved or improved before the code is merged into the main code repository. In this context, those issues can also be considered as Potential Technical Debt (PTD).

The definition of PTD was first proposed by Schmid [31], which describe it as a risk that can potentially become a problem. In this work, we adopted the definition of PTD as the early stage of TD in code reviews, because PTD will evolve into TD if it has been introduced into software systems without being resolved, otherwise it would not have any real impact on the quality of the software. Considering the fact that PTD is a major source of TD, it is critical to identify and manage PTD as early as possible, which can make developers aware of the existence of PTD, and then decide when and how to resolve the PTD to avoid the occurrence of TD. Nowadays, many research and industrial tools have been provided (such as static code analysis tools) to detect TD, but these tools lack consensus [20] and a large amount of PTD cannot be detected by existing tools [19].

Considering this, **we focused on** the concept of PTD and conducted an exploratory study to understand PTD and its distributions, classification, and resolution, as identified in code reviews. We first collected code review comments from two widely known projects: the Nova project of OpenStack and the Qt Base project of Qt. We then manually labelled and analyzed each review comment to study how much PTD is identified in code reviews, what types of PTD are identified, what actions are taken by developers to resolve the PTD, and how long it takes for developers to pay back the TD. We analyzed 163 PTD-related review comments acquired from a total of 2,030 review comments from the two projects.

The remaining sections of this paper are structured as follows: Section 2 surveys the related work on TD and code reviews. Section 3 describes the research design of this study. The results of our study are presented in Section 4, followed by a discussion of the implications of our results in Section 5. Section 6 clarifies the threats to the validity. Finally, we conclude this work in Section 7.

2 RELATED WORK

2.1 Technical Debt

According to Li *et al.* [22], TD occurs when technical shortcuts are taken by developers to gain short-term profits that are harmful to the system in the long term. In recent years, the research community has extensively worked on different aspects of TD. Zazworka *et al.* compared the results between manual identification of TD by researchers and automatic detection of TD by tools [38]. Their result shows that the tools employed are especially useful for identifying defect debt but cannot help in identifying many other types of debt, so involving human effort in the identification process is necessary. De Toledo *et al.* conducted a case study on a project from a large company to investigate architecture TD in microservices architecture [6]. They combined the existing documentation and interviews to detect problems, solutions, and risks, providing a list of architectural issues that generate TD, which is useful for practitioners that want to avoid or repay TD.

Moreover, many studies focus on identifying TD from various types of sources. For instance, Potdar *et al.* used code comments in four large Open Source Software (OSS) projects to identify SATD. They found that developers with higher experience tend to introduce most of the SATD [29]. Silva *et al.* focused on identifying TD in pull requests, and their results reveal that 30.3% of the pull request rejected are due to TD, and the most frequently TD

encountered is design TD [32]. Li *et al.* explored SATD in issue trackers, and found that the majority of TD is paid off [21].

Inspired by the body of work discussed above, we were motivated to investigate PTD, a source of TD [31]. The difference between our work and existing work is that we identified and analyzed PTD in the context of code review, which involves reliable human identification process (i.e., reviewers could consider taking full consideration of the contextual information and thus be better positioned to detect PTD in the code). Recently, an empirical study was conducted by Kashiwa *et al.* to examine the relationship between SATD and code review process, which is the most related work to us [17]. However, in that work, the authors mined SATD only from the code comments that introduce SATD during the code review process. Moreover, the paper did not investigate the repayment or resolution of TD in code review. Specifically, we considered identifying PTD from the review comments of code reviewers and developers to understand PTD and its distribution, classification, and resolution in code reviews.

2.2 Code Review

Code review is a typical process in software development. Many studies have explored modern code review processes in practice. For instance, McIntosh *et al.* studied the relationship between code review and post-release defects [26]. Their results empirically confirm the intuition that poorly-reviewed code has a negative impact on software quality. Baysal *et al.* investigated the influence of non-technical factors (such as patch size, organizational and personal dimensions) on code review [2]. Their results show that organizational and personal factors influence review timeliness, as well as the likelihood of a patch being accepted.

There are also studies focusing on investigating a variety of artifacts in code reviews. For instance, Zanaty *et al.* set out to study the frequency and nature of design discussions in code reviews [36]. They collected and analyzed the review comments of the OpenStack's Nova and Neutron projects, and their results indicate that design concerns are not commonly discussed in code reviews and most design related comments are constructive. Nanthaamornphong *et al.* examined comments from code reviewers that identified code smells in two open source projects (i.e., OpenStack and Wikimedia) [27]. Their preliminary results show that code reviewers comment on only a small number of code smells. In order to further investigate code smells in code reviews, Han *et al.* conducted an empirical study using the same projects as used by Zanaty *et al.* [36] (Nova and Neutron) to study how code smells are identified in code reviews, and the actions taken against the identified smells [13, 14]. Their result shows that reviewers usually provided constructive feedback, including fixing (refactoring) recommendations to help developers remove smells. Mello *et al.* focused on exploring the influence of human factors for identifying code smells in code reviews [5]. Their results reveal that human factors play a key role in the precise identification of code smells, and reviewers with professional background could reach a high precision of smell identification.

The above work motivates us to identify and analyze PTD in the context of code reviews. Their selection of communities and projects lay the foundation for this study.

3 RESEARCH DESIGN

3.1 Research Questions

The goal of this work, formulated through a Goal Question Metric approach [1], is to investigate the existence of PTD in code reviews **for the purpose of exploration with respect to** PTD and its distribution, classification, and resolution **from the point of** developers and code reviewers **in the context of** OSS projects. To achieve this goal, we formulated the following five research questions (RQs), as follows:

RQ1. What is the proportion of review comments that indicate PTD in code reviews?

Rationale: As an exploratory study on PTD in code reviews, this RQ aims at providing a basic view of the distribution of PTD, which can help improve developers' awareness of PTD.

RQ2. What are the types of PTD identified in code reviews?

Rationale: Different types of PTD may have different impact on development, and they may also have different priorities to be resolved. Therefore, this RQ intends to provide an understanding regarding the classification of PTD in code reviews. Moreover, by providing the distribution of different types of PTD in code reviews, the results of this question can also help developers be aware of the most common problems that they are confronted with.

RQ3. How much of the identified PTD has been resolved and how much of the identified PTD has evolved into TD?

Rationale: When PTD is identified in the code reviews, developers could decide whether resolve it in the later patchsets of the code change or temporarily ignore it. In the latter case, the PTD will evolve into TD because it is not resolved when code change is merged into the main code repository, which can have a negative impact on the software system. To help stakeholders better manage PTD, it is necessary to first quantify how much of the identified PTD has been resolved or evolved into TD.

RQ4. How long does it take for developers to resolve identified PTD in code reviews?

Rationale: For the PTD that has been resolved, we would like to know how long it takes for developers to resolve the identified PTD. Answering this RQ can help better prepare developers for fixing similar issues in the future. In addition, since the severity and resolution difficulty of various types of PTD can be different, we further investigate the difference in time taken when resolving different types of PTD.

RQ5. What practices are used by developers to resolve PTD that has been identified in code reviews?

Rationale: By answering this RQ, we aim to investigate the practices employed by developers to resolve PTD. Such information could help stakeholders (e.g., developers) better manage and resolve PTD in the development process.

3.2 Data Collection

This study analyses PTD in code reviews collected from two large projects of the OpenStack¹ and Qt² communities. OpenStack is a set of software tools for building and managing cloud computing

platforms, and is supported by many large companies. Qt is a cross-platform application and UI framework developed by the Digia corporation, but welcomes contributions from the community at large. Since these two communities have made a big investment in code reviews for several years [15] and are widely used in many studies related to code reviews [12, 34], we deemed them to be appropriate and representative for our analysis. The OpenStack and Qt communities are composed of several projects, and we selected one of the most active projects from each community (based on the highest number of closed code changes), i.e., Nova³ from OpenStack and Qt Base⁴ from Qt.

OpenStack and Qt adopt Gerrit⁵, a web-based code review platform built on top of Git, to support their code review process. By using the RESTful API provided by Gerrit, we collected all the code changes of the two selected projects that were last updated in 2020. We then extracted all available review comments of these code changes and stored the data in a local file for further analysis. In total, we collected a dataset of 1,840 code changes and 13,982 review comments from Nova, and 8,954 code changes and 36,924 review comments from Qt Base.

3.3 Data Labelling and Extraction

Manually analyzing all the review comments from Nova (13,982) and Qt Base (36,924) is a time-consuming task. Considering this, we decided to randomly select a representative sample of review comments for each project based on 3% margin of error and 95% confidence level [16]. This leads to the selection of 992 and 1,038 review comments from Nova and Qt Base, respectively.

In the **data labelling** step, we manually read through all review comments to label whether they are related to PTD. For the PTD-related review comments, we classified and labelled their types using the TD taxonomies provided by Maldonado *et al.* [25], Liu *et al.* [24], and Li *et al.* [22]. The reason we chose the description of TD types as the basis for classifying PTD is that PTD will evolve into the corresponding type of TD when it is injected into the software systems without being resolved.

To mitigate any selection bias, we conducted a pilot labelling of the PTD-related comments with all the 992 code review comments randomly selected from Nova by the first and third authors, independently. The disagreements on the labelling results were discussed and resolved with the second author to get a consistent understanding of the criteria of data labelling, i.e., the review comment should be clearly related to PTD and meet the definition of one of the TD types, which are illustrated in Section 4. After the first and third authors achieved a consensus on the labelling of all the review comments of Nova, they then further labelled the 1,038 code review comments randomly selected from Qt Base, independently. Any disagreements on the labelling results were still discussed and resolved with the second author. We measured the inter-rater reliability and calculated the Cohen's Kappa coefficient [3] as a way to verify the consistency on the labelled review comments of Qt Base between the first and third authors. The Cohen's Kappa

¹<https://www.openstack.org/>

²<https://www.qt.io/>

³<https://github.com/openstack/nova>

⁴<https://github.com/qt/qtbase>

⁵<https://www.gerritcodereview.com/>

coefficient is 0.81, indicating that the two coders reached a decent agreement on labelling PTD-related comments.

In the **data extraction** step, for answering RQ3, RQ4, and RQ5, we analyzed the contextual information of each identified PTD-related review comment, including the code review discussions and associated source code to determine whether the identified PTD was resolved. Specially, we regarded a PTD as resolved when the resolution situation fits into the following three categories:

- 1) Changes were made in the code by the developer(s) to resolve the PTD before the code change is merged (see Figure 1).
- 2) Developer(s) clearly mentioned that the PTD has been resolved in another follow-up code change.
- 3) The code changes that introduced the PTD were abandoned so that the PTD would not have any effect on the main code repository. In other words, the PTD no longer exists.

For each resolved PTD, we extracted and recorded its resolution information. Specifically, for the existence time of PTD, we calculated the time interval between the identification time (the timestamp of when the corresponding review comment was written) and the resolution time (the timestamp of when developers committed patchset to resolve the PTD or when developers abandoned the code change that introduced the PTD). The resolution practices were analyzed based on the extracted information using Constant Comparison [11]. This data extraction process was conducted by the first author and the results were verified by two other co-authors. Conflicts were discussed and addressed by the three authors. Table 1 presents the data items to be extracted and their corresponding RQs in this study, which have been provided online [10].

Table 1: Data Items to be Extracted from Review Comments

Data Item	Description	RQ
PTD-Related	Whether the review comment is related to PTD (i.e., Yes or No).	RQ1
PTD	The PTD text extracted from the message.	RQ1
Who Identifies PTD	The role of person who identifies PTD in the review comment (i.e., Reviewer, Developer, or Developer after discussion).	RQ1
PTD-Type	The type of the identified PTD.	RQ2
Resolved	Whether the identified PTD is resolved (i.e., Yes or No).	RQ3
Resolution Evidence	Any information about where the identified PTD is resolved in the code.	RQ3
Identification Time	The time when the review comment that identifies PTD is added.	RQ4
Resolution Time	The time when identified PTD is resolved.	RQ4
Existence Time	The time interval between identification time and resolution time of the identified PTD.	RQ4
Resolution Practice	The practice that developers take to resolve the identified PTD.	RQ5

4 RESULTS AND ANALYSIS

RQ1: What is the proportion of review comments that indicate PTD in code reviews?

Table 2 presents an overview of review comments per project. In general, we identified a total of 163 review comments that indicate PTD. Compared with the number of all the review comments we analyzed, we can find that PTD is not so prevalent in code

reviews, only accounted for 8.0% on average. Concretely, there are 90 PTD-related review comments that are identified in code reviews for Nova (out of 992, 9.1%). The number of PTD-related review comments identified from Qt Base is less than that of Nova, and the proportion is a bit lower (73 out of 1,038, 7.0%).

We further investigated how much of PTD is identified by code reviewers and developers. We categorized the role of the person who identified PTD in code reviews into three situations, i.e., Reviewer, Developer, or Developer after discussion. We found that developers sometimes take the initiative to write down the review comments and record the issues that contain PTD in code reviews. However, it is not so prevalent with only 4 instances (out of 163, 2.5%). In contrast, most of PTD was identified by reviewers (143 out of 163, 87.7%). Moreover, PTD can also be identified in the response comments of developers. In this situation, developers can be aware of the existence of PTD through the discussion with reviewers (16 out of 163, 9.8%). As shown in the example below, the developer forgot to remove the redundant code after debugging and was aware of the code issue after it was pointed out by the reviewer, which indicates a code PTD. In a word, the identification of the overwhelming majority of PTD (97.5%) owes to the help of reviewers in the code review process. This finding indicates that developers may not realize that they introduce the PTD during the development process. It also shows the important role that reviewers play in the code review process for identifying the PTD.

Link: <http://alturl.com/o24nq>

Project: Nova

Reviewer: “I don’t get this... But then why we register them for this test.”

Developer: “Sorry, this was debug ... I added this to make it run consistently in isolation while I figured out what was going on. I meant to remove this and forgot.”

RQ1 Summary: PTD can be identified in code reviews but is not prevalent. Specifically, only 8.0% of review comments contain PTD on average. Moreover, reviewers play a significant role for identifying PTD in the code review process.

Table 2: Overview of Review Comments per Project

Project	Review Comments	PTD-related Review Comments	%
Nova	992	90	9.1
Qt Base	1038	73	7.0
Total	2030	163	8.0

RQ2: What are the types of PTD identified in code reviews?

As mentioned in Section 3.3, we adopted the classification of TD types provided in Maldonado *et al.* [25], Liu *et al.* [24], and Li *et al.* [22] as the basis for classifying PTD in this work. Moreover, the aforementioned three studies also provide detailed *causes* of TD, which are helpful for understanding and classifying PTD.

In this work, we identified six types of PTD in code reviews, and we provide examples of review comments for each PTD type below.

1. Design PTD refers to the technical shortcuts that are taken in detailed design, which lead to sub-optimal design of the code.

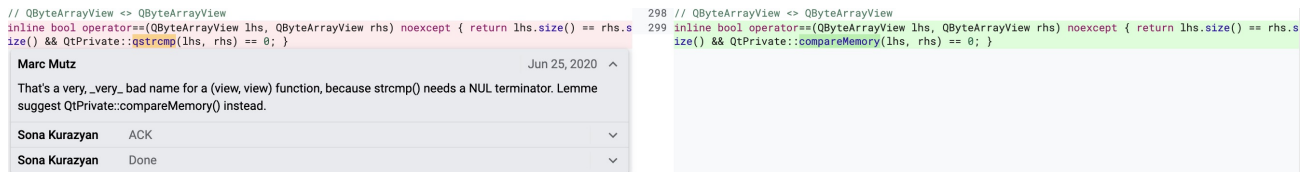


Figure 1: An example of a *renaming bad name* operation to resolve PTD after review (the change is highlighted)

Various reasons could lead to design PTD, e.g., lack of abstraction, poor implementation, workarounds, or temporary solutions.

We found that *poor implementation of code* [24, 25] is the most important reason that leads to design PTD in code reviews. In this case, the current implementation may satisfy the functionality, but developers are unaware that the design of the code is sub-optimal.

Link: <http://alturl.com/6srnp>

Project: Nova

Reviewer: "Don't do this at query time. There are not likely to be many of these and apparently these queries are `_slow_`. Instead, do it in the Python code below."

Link: <http://alturl.com/s63p7>

Project: Qt Base

Reviewer: "This leads to very poor code expansion. `is_shared` should either be LSB or MSB. The advantage of being MSB (last in declaration order for little-endian) is that it becomes the sign bit."

The reviewers were negative about current implementation and recommended specific implementation to improve the design of the code. Due to the lack of experience or context knowledge of the projects, developers might write not-quite-right code without considering whether the implementation is optimal. Such poor implementation can lead to various quality issues in the software systems (e.g., low performance and extensibility in our examples).

Code duplication [24] is another reason that can result in design PTD in code reviews. In order to reduce the workload in development, developers aim at reusing code as much as possible. To achieve this goal, for example, methods with single responsibility can be extracted and reused rather than duplicating the same code in multiple places. In the below example, the reviewer thought that the developer should write a helper function in the code to meet the DRY (Don't Repeat Yourself) principle. Code duplication could increase the maintenance burden since developers might need to change the code in multiple places to reflect on a single change elsewhere in the code. This in turn can lead to bugs being introduced if part of code is left unchanged.

Link: <http://alturl.com/wgyyu>

Project: Nova

Reviewer: "Shouldn't we have kept the helper function (and modify and rename it) for doing this part to keep this DRYer?"

2. Defect PTD indicates that the code behaves in an unexpected way since there are defects/bugs in the code. As shown in the example below, the developer mistakenly thought that the code works without any issues. However, the reviewer pointed out that the wrong code is potentially flaky (behaves non-deterministically), indicating that there is a defect in the code.

Link: <http://alturl.com/a2pap>

Project: Qt Base

Reviewer: "We shouldn't call `qui` methods from non-gui threads I guess."

Developer: "Not sure what's different with `QStatusBar`, but it works without any issues ..."

Reviewer: "Well, you are apparently calling `showMessage` from the different thread than `statusBar` lives in. `QStatusBar` is not reentrant, neither `showMessage`. Works just by coincidence, but it's wrong."

3. Documentation PTD indicates the *lack of code comments* [22] or *inadequate documentation* [22, 24] that could explain the corresponding part of the system. We found that reviewers usually expect developers to provide clear descriptions of the code they write by providing code comments. Nevertheless, developers sometimes forget to write code comments, which leads to the documentation PTD. For example:

Link: <http://alturl.com/p75ju>

Project: Nova

Reviewer: "supernit - I know it's pretty straight forward but a quick comment outlining what the test is doing would be useful."

Software documentation is a critical activity in software engineering [18]. Proper documentation can not only make it easier for other developers to reuse the code, but also decrease the costs and difficulties of maintenance. Developers should attach great importance to documentation and write high quality documentation for the code they write.

We also found that *outdated documentation* [22] will lead to documentation PTD. In the below example, the reviewer pointed out that the function documentation should be updated since the function has been changed, which indicates that the old code comment was outdated. Outdated documentation can mislead the readers so that they may fail to grasp the intent of the code. If other developers are misled by outdated documentation when they plan to reuse the code, it may lead to code issues with unexpected costs.

Link: <http://alturl.com/any97>

Project: Nova

Reviewer: “This function now returns one single ResourceProvider hence the function doc needs to be updated”

4. Requirement PTD indicates the incompleteness of method, class, or program.

The reviewers would point out the issues if they think the code is incomplete. Due to time pressure or other constraints, developers sometimes did not immediately follow reviewers’ recommendations but chose to leave a TODO comment to remind themselves to do it later. Therefore, this can be a requirement PTD until the TODO comment is completed. For example:

Link: <http://alturl.com/8czn4>

Project: Nova

Reviewer: “I would raise an exception if a key already defined to help avoid duplicated validators.”

Developer: “Yup, added a TODO to do this. This whole thing is a bit janky at the moment and needs to be shuffled around.”

Moreover, we observed that new requirements can be identified in the discussions between reviewers and developers. In the below example, we can see that a new functional requirement is mentioned by the developer (i.e., “a future improvement on *QFutureInterface* to make it movable”). In this context, the new requirement is clearly known by the developer, and hence should be scheduled in the future development plan. Therefore, this can also be a requirement PTD until the new requirement is fulfilled.

Link: <http://alturl.com/eu4zw>

Project: Qt Base

Reviewer: “`d(std::exchange(other.d, ))?`”

Developer: “Oh, good suggestion. There’s no move semantics in `d` at the moment (unsure how `std::exchange` requires `MoveConstructible` to be true), but that’s a topic of future improvement: making `QFutureInterface` movable and simplifying both this part and swap.”

5. Test PTD indicates the need for implementation or improvement of the current tests. We found that *lack of tests* [22, 24] is the main reason that leads to test PTD.

As shown in the below examples, the review comments related to test PTD explicitly convey the meaning that tests are insufficient and the developers should add new tests or improve existing tests. Lack of tests makes it hard for developers to find bugs and defects buried in the code, and then increases the number of defect PTD.

Link: <http://alturl.com/obj6s>

Project: Nova

Reviewer: “This seems not covered by test case.”

Link: <http://alturl.com/fckqu>

Project: Qt Base

Reviewer: “there must be tests for terminate & kill - these seem more relevant to this matter, imo. feel free to fix the test in a separate commit, obviously”

6. Code PTD refers to the poorly written code that violates best coding practices or coding conventions.

We found that *violating code conventions* [22] is one of the reasons that can lead to code PTD identified in code reviews. Taking bad naming as an example, following a naming convention plays an important role in making code readable. In the below example, the code reviewer identified an issue of naming variables with a single character (i.e., `f` and `l`). A meaningful naming should tell you what the variable or function stands for. Otherwise, this might confuse those reading the code. That is why the reviewer recommended to rename `f` and `l` to `firstIter` and `lastIter`, respectively.

Link: <http://alturl.com/9zog3>

Project: Qt Base

Reviewer: “Please try harder; single-char identifiers are unreadable, and `l` is particularly bad for reading. `firstIter/lastIter` will work sufficiently well.”

We also found that *low-quality code* [22] identified in code reviews can lead to code PTD. There are various reasons that decrease the quality of the code, e.g., bad copy/paste activities, redundant code, using magic value, and so on. For instance, developers sometimes copy and paste code snippets without thinking about whether the code is appropriate to reuse in their context, which decreases the quality of the code in general, as in the following example:

Link: <http://alturl.com/y6zmn>

Project: Nova

Reviewer: “I suspect it was more a case of copying prior art from `‘nova.tests.functional.regressions’`”

Moreover, as shown in the example below, we can see that developers sometimes use magic value in their code, which decreases the quality of the code because using magic value can degrade the readability of the code and make it harder to maintain.

Link: <http://alturl.com/vwdcw>

Project: Qt Base

Reviewer: “Please duplicate the code from above with the comment to avoid the 1 magic value: ...”

Table 3 presents the distribution of different types of PTD in code reviews ordered by their numbers. We can find that code PTD is by far the most frequently identified PTD, with exactly 55 instances (33.7%). The second most frequent type is documentation PTD, which accounts for 27.6%, nearly the same as code PTD. There are 21 review comments (12.9%) that identified test PTD, followed

by design PTD making up 12.3%. Requirement PTD and defect PTD are the types with the lowest proportions (less than 10%).

Table 3: Distribution of Types of PTD in Code Reviews

Type	Number	Percentage
Code TD	55	33.7%
Documentation TD	45	27.6%
Test TD	21	12.9%
Design TD	20	12.3%
Requirement TD	12	7.4%
Defect TD	10	6.1%

RQ2 Summary: We have identified six types of PTD in code reviews, i.e., design, defect, documentation, requirement, test, and code PTD. We found that the majority of the PTD identified in code reviews is code PTD (33.7%) and documentation PTD (27.6%).

RQ3: How much of the identified PTD has been resolved and how much of the identified PTD has evolved into TD?

Table 4 presents the resolution situation of the PTD identified in code reviews. Of the 163 PTD, the majority (132, 81.0%) has been resolved by the developers after the review. In detail, the proportions of the TD that was resolved in Nova and Qt Base are almost the same, which are 81.1% and 80.8%, respectively. This finding indicates that developers tend to resolve the PTD under the suggestions of reviewers to help increase the possibility of code changes passing the code review. However, 19.0% of PTD remains unresolved. Therefore, once the code change is merged, the unresolved PTD would be injected into the software system in the form of TD (i.e., evolves into a real TD), which might have serious negative impact on the main code repository.

Table 4: Payment Situation of TD-related Review Comments per Project

Project	Resolved	% Resolved	Remaining	% Remaining
Nova	73	81.1%	17	18.9%
Qt Base	59	80.8%	14	19.2%
Total	132	81.0%	31	19.0%

Moreover, we also investigated the resolution situation of each PTD type as shown in Table 5. From the table, we can see that test PTD is the debt with the highest percentage of being paid by developers, which corresponds to 90.5%. We can also learn that the resolution proportion of code PTD is also very high (close to test PTD) even if the number of code PTD is the highest. This finding indicates that developers show their great emphasis on code TD, and they resolve code PTD with an active attitude. The resolution proportion of documentation PTD is also quite high, which is 86.7%. This may be attributed to the explicit targets of how to resolve documentation PTD, e.g., developers could supplement or update the corresponding code comments after reviewers point out what comments are missing or which documentation is outdated. However, we can notice that only 16.7% of requirement PTD has been resolved by developers, which is much lower than other types of PTD. Potential reasons could be that: (1) a requirement may

rely on another requirement, and thus it cannot be implemented in time. As shown in the response of a developer “*would you be ok with me just adding a TODO in the code to extend the virt driver rebuild function if support is added for cyborg to a driver that uses it?*”, the developer could only extend the rebuild function after adding support for cyborg; (2) many requirements are related to the further improvement on methods, classes, or program. Therefore, compared with other issues identified in code reviews, such requirements may be put in lower priorities by developers and be scheduled and fulfilled in the future.

Table 5: Resolution Situation of Each PTD Type

Type	Num	Resolved by Developers	%
Code PTD	55	49	89.1%
Documentation PTD	45	39	86.7%
Test PTD	21	19	90.5%
Design PTD	20	16	80.0%
Requirement PTD	12	2	16.7%
Defect PTD	10	7	70.0%

RQ3 Summary: On average, about 81.0% of the PTD identified in code reviews was resolved by developers, and the remaining 19.0% of the PTD evolved into the TD. Moreover, most of test PTD, code PTD, and documentation PTD have been resolved by developers.

RQ4: How long does it take for developers to resolve identified PTD in code reviews?

According to the results of RQ3, a total of 132 PTD identified in code reviews was resolved by developers. To show the distribution of time for payment, we plotted the results in a histogram (as shown in Figure 2). From the figure, we can see that more than half of PTD has been resolved in less than one day, which corresponds to 69 (52.3%) of 132 PTD-related review comments. In 34 (25.8%) PTD-related review comments, developers chose to resolve the PTD in more than one day but less than one week after review. That is to say, developers can resolve the majority of the PTD in less than a week (103 out of 132, 78.0%). 17 (12.9%) of the PTD takes developers 2-4 weeks to resolve, and only few PTD (9.1%) costs developers more than a month to fix.

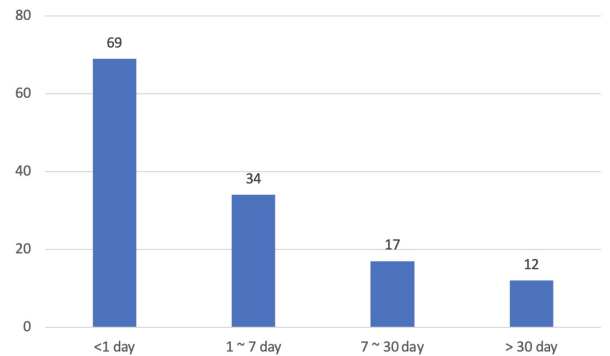


Figure 2: Distribution of Time for Payment

Moreover, we also investigated the resolution time of each PTD type. As shown in the Table 6, we listed the minimum and

median time for resolving different types of PTD. We find that the minimum time taken for resolving all types of PTD except requirement PTD can be very short - only near one hour. Although the number of the resolved requirement PTD is limited in our cases, the longest minimum resolution time of requirement PTD (289 hours) still can reflect the hardness when developers were confronted with requirement issues. According to the median results, we can find that the documentation PTD (median = 18 hours) was resolved faster than other types of PTD, which suggests that the documentation PTD would be easier to be addressed by developers. We believe this is due to the nature of documentation PTD. Usually the targets of how to resolve documentation PTD are rather explicit as described in the results of RQ3, and there is no need to modify the code when resolving the documentation PTD. Code PTD (median = 22 hours) and Design PTD (median = 27 hours) are also quite fast to resolve, which costs around a day. The resolution time of Defect PTD (median = 48 hours) and Test PTD (median = 116 hours) is much longer than the aforementioned three types of PTD (i.e., documentation, code, and design PTD). Same as the minimum result of requirement PTD, the median result of requirement PTD (325.5 hours) also reveals the fact that developers need to take more time to consider about how to further implement and complete the incomplete method, class, or program.

Table 6: Time taken for resolving different types of PTD

Type	Minimum (hour)	Median (hour)
Code PTD	1h	22h
Documentation PTD	1h	18h
Test PTD	1h	116h
Design PTD	1h	27h
Requirement PTD	289h	325.5h
Defect PTD	1h	48h

RQ4 Summary: On average, about 78.0% of the resolved PTD was resolved by developers within a week. Moreover, documentation PTD takes less time to resolve (18 hours on median) while requirement PTD costs the longest time to resolve (325.5 hours on median).

RQ5: What practices are used by developers to resolve PTD that has been identified in code reviews?

For answering this question, we analyzed the code review discussions and associated source code of each PTD, and identified the practices used by developers to resolve the PTD identified in code reviews. Five resolution practices are then identified, which are presented below:

- 1) **Code refactoring** is the process of restructure the code with the aim of improving the design, structure, or the non-functional properties (e.g., readability, maintainability, or complexity) while preserving its functionality.
- 2) **Documentation improvement** is the process of improving the quality of documentation, including supplementing code comments, restructuring the statements of documentation, or removing the outdated documentation.
- 3) **Testing improvement** is the process of adding new tests or improving the existing tests.

- 4) **Bug fixing** is the process of correcting the known defects/bugs in the software system.
- 5) **Code change abandonment** refers to the cases where developers may abandon the code changes. In such cases, the code that contains PTD will not be merged in the main code repository so that the PTD will not have an effect on the project. Considering this, we regarded it as a special resolution practice. However, the intentions of developers to abandon the code changes may not be necessarily due to PTD. Therefore, we treated it as a resolution practice only if developers did not use one of the other four resolution practices and the code change was abandoned.

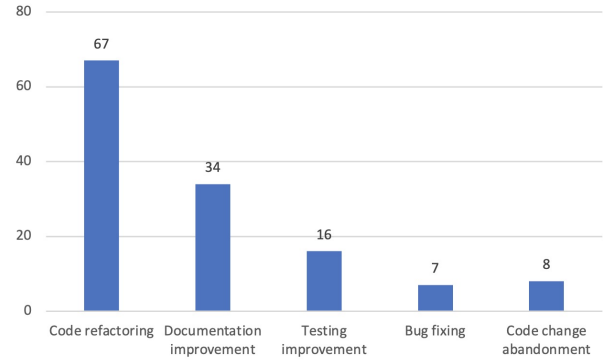


Figure 3: Distribution of Payment Practices

As demonstrated in Figure 3, we can see that the most frequent payment practice used by developers is **Code refactoring** (67 out of 132, 50.9%), which is almost two times greater than the second most used resolution practice in our cases. Some of the examples replied by developers to the review comments are: “*This code would need a refactoring to become clearer IMHO.*”⁶ and “*Refactored in PS8.*”⁷. Code refactoring is a base practice that can be used for a variety of purposes, such as improving the design of an existing code base [8], improve the internal quality of the code [22], and so on. Therefore, it plays an important role in resolving code PTD and design PTD identified in code reviews.

With the high proportion (27.6%) and high resolution percentage (86.7%) of documentation PTD, the following resolution practice used by developers in code reviews is **Documentation improvement** (34 out of 132, 25.8%). An example of such a case is shown below, where the reviewer recommended adding a comment for the pattern and the developer followed the recommendation of the reviewer and supplemented the missing code comment. It is important for developers to improve the documentation in development process. Better documentation (e.g., source code comments) could make the information easier to access for developers, help the readers better and quicker to grasp the intention of the source code (such as when other developers might want to reuse your code), lighten the burden of maintenance.

⁶<http://alturl.com/k4qhi>

⁷<http://alturl.com/8ptv7>

Link: <http://alturl.com/siws8>

Project: Qt Base

Reviewer: “I like this pattern, but it deserve a comment:
// prev is a pointer to the “next” element within the previous
node, or to the “firstObserverPtr” if it is the first node.”

Developer: “Agreed. Thanks for the comment - added.”

In addition, 16 (12.1%) PTD was resolved by **Testing improvement**, such as the responses of developers in the following examples: “Add one test case for the case⁸” and “I’ll add a simple test to assert the capability...⁹”. By testing, developers could find bugs or defects in advance, and fix them to guarantee the quality of software. Developers could also evaluate the specific properties of software by testing to make the software more robust and reliable. Also, testing could be helpful to avoid the injection of TD in software systems [28]. Closely related to **Testing improvement**, **Bug fixing** is mainly used to resolve or correct the defect PTD after finding the defects or bugs. It accounts for 5.3% (7 out of 132) in our cases.

In the rest (8 out of 132, 6.1%) PTD, the developers abandoned the code changes and the PTD was thus disappeared with the code. **Code change abandonment** is considered as a special resolution practice in our study since the PTD is “resolved” by discarding the code changes. However, it will lead to extra cost of human resources, time, and budget if the code changes are abandoned in the working progress. Therefore, further investigation on the reasons why developers abandon the code changes is needed.

RQ5 Summary: We have identified five resolution practices to resolve the PTD identified in code reviews. Among them, code refactoring is the main practice, which is used to resolve the majority of the PTD.

5 IMPLICATIONS

5.1 Implications for Practitioners

For project managers, we argue that code review, as a good practice for software quality assurance in software development [35], could be used as one of the trustworthy mechanisms to detect PTD in the development process, since our study reveals that reviewers are able to spot PTD that developers are not aware of and let developers resolve PTD at an early stage. Moreover, in our study we find that although the majority of the identified PTD has been resolved by developers, a significant proportion of the identified PTD (19.0%) was still injected into the software systems without being resolved, which may severely degrade the code quality and impact the maintainability of the systems, especially we observed that only 16.7% of requirement PTD was resolved as shown in the results of RQ3. Hence, there is a need for establishing an effective mechanism for PTD management. For the identified PTD, developers should try their best to resolve it before the code is merged into the main repository. If the PTD could not be resolved due to time pressure or other constraints, it should be explicitly recorded and scheduled to be resolved in the future.

For developers, our study finds that most of PTD is identified by reviewers, which implies that developers may write not-quite-right code due to oversight or lack of experience. Therefore, we provide the following suggestions to avoid the occurrence of PTD and resolve PTD:

1) The results of RQ2 shows that code PTD is the majority of PTD, which indicates that developers are more likely to write not-quite-right code. To avoid the occurrence of code PTD, developers should follow closely coding conventions and the best coding practices, such as using meaningful naming for variables and functions in the code, avoiding using magic value, copy/paste activities, and so on.

2) For documentation PTD, developers should actively write necessary documentation (e.g., code comments), and pay attention to the corresponding documentation when modifying the code to avoid making the documentation outdated and invalid. According to the result of RQ4, documentation PTD costs the less time to resolve compared with other types of PTD, which implies that it is easier to resolve compared to other types of PTD.

3) For test PTD, based on the main reasons that lead to Test PTD in the results of RQ2, developers could consider more about the test coverage of their code and add sufficient test cases in the development process.

4) For design PTD, as shown in the results of RQ2, poor implementation is the main reason that could result in design PTD. Developers should learn from the suggestions of reviewers and gain knowledge, such as the ideas and insights towards the design issues, which could help them implement the design in code in a better way.

5) For requirement PTD, there is still a large proportion of requirement PTD remaining unresolved. Considering the difficulty of resolving requirement PTD, developers could first record them which can be managed and resolved in the future.

6) For defect PTD, developers should timely resolve it, otherwise it might affect the correctness of the system and incur interest [23] when the code contains defect PTD is merged. For the unresolved defect PTD, developers could explicitly record it (e.g., in the commit message or code comment) to inform other developers.

5.2 Implications for Researchers

In this work, we focused on PTD, which is an important source of TD. PTD refers to the technical compromises (e.g., sub-optimal implementations) in the development process concerned with the aspects from implementation (i.e., at the code level) to design, and even documentation, requirements, and testing. PTD will evolve into TD when it is injected into the system (e.g., code is merged into the main repository) without being resolved. In our study, we aimed to identify PTD in code review, a process of manually reviewing and verifying code during development. As an exploratory study regarding PTD in code reviews, this work provides researchers with a basic view of PTD by investigating its distribution, classification, and resolution. We believe that the knowledge of PTD could help researchers gain new insights towards the understanding and management of TD. For example, according to the results of RQ3, the resolution proportion of code PTD is very high while the number of code PTD is the highest. This finding indicates that developers show their great emphasis on code PTD, and they resolve

⁸<http://alturl.com/37ssk>

⁹<http://alturl.com/txfec>

PTD with an active attitude. A recent study, that investigated the practitioners' intent for self-fixing TD [33], came to a similar conclusion that code TD is the main concern compared with other types of TD. Our results of RQ5 reveal that code refactoring is the main practice to resolve PTD. Previous studies on the payment of TD show that code refactoring is also the main practice to pay back TD [9, 28]. In addition, some TD payment practices are also employed as the resolution practices of PTD, such as *Documentation improvement* corresponds to the payment practice *Updating system documentation* identified in [9], and *Testing improvement* is the same as the payment practice *improve testing* in [28].

Further research could be done towards understanding and managing PTD. Our data was extracted from code review comments. Mining PTD from other sources, such as pull requests together with issue trackers on GitHub, can provide more valuable and comprehensive data for further analysis and comparison. Researchers could pay more attention to the design of automatic tools to identify and classify PTD in review comments, which can help both developers and reviewers better track and deal with PTD in code reviews. In addition, it is also necessary to investigate the reasons for why developers chose not to resolve such PTD before it was injected into the software system, which may help to come up with effective PTD management strategies.

6 THREATS TO VALIDITY

Construct Validity reflects on the extent of consistency between the operational measures of the study and the RQs [30]. A potential threat in this study involves whether the collected review comments were correctly labelled and analyzed by the researchers. In this work, we depended on human activities, which would induce personal bias. To reduce this threat, we labelled the review comments by two researchers independently. Any disagreements were discussed and addressed with a third researcher. Moreover, we conducted a pilot study to make sure that the two researchers achieved a consensus understanding of PTD-related comments. The data extraction process was conducted by one researcher, and the results were reviewed and checked by other two researchers. Another potential threat is whether the dataset is sufficient enough to obtain reasonable conclusions. For RQ1, and RQ2, we randomly selected a representative sample size based on 3% margin of error and 95% confidence level [16]. We believe that this measure can partially alleviate this threat. However, there are only 163 PTD-related comments for answering RQ3, RQ4, and RQ5. We admit that the sample size may threaten the validity of our results. Therefore, we conjecture that we could obtain more convincing results by extending this work in a large dataset from more communities and projects, which is also our next step.

External Validity refers to the degree to which our study results and findings can be generalized in other cases (e.g., other projects in OpenStack and Qt, or projects in other communities). We considered the OpenStack and Qt communities since these two communities have made a serious investment in code reviews for several years and are widely used in many studies related to code reviews. As for the selection of projects, we select the most active and major project for each community as our investigated projects. Therefore,

we argue that the communities and projects are representative and can increase the generalizability of our study results.

Reliability refers to the replicability of a study for obtaining same or similar results. To improve the reliability, we made a research protocol with detailed procedure, which was discussed and confirmed by all the authors. Besides, all of the empirical steps in our study, including the data mining process, manual data labelling, and manual data extraction and analysis, were conducted and discussed by three authors. Furthermore, the dataset and analysis results of our study have been made publicly available online in order to facilitate other researchers to replicate our study easily [10]. We believe that these measures can partially alleviate this threat.

7 CONCLUSIONS

This paper reports on an exploratory study that investigated PTD and its resolution in code reviews. We chose two widely known communities (i.e., OpenStack and Qt) since they have invested heavily in their code reviews for several years and provided reliable peer review activity data. We then randomly collected and analyzed 2,030 review comments from two active projects (i.e., Nova of OpenStack and Qt Base of Qt).

According to our results, PTD is not prevalently identified in code reviews, and when identified, code and documentation PTD are the most frequently identified PTD among six types of PTD (i.e., design, defect, documentation, requirement, test, and code PTD). This finding indicates that developers should care more about their code quality along with the corresponding documentation when implementing and modifying the code. We also found that around 81.0% of the PTD identified in code reviews has been resolved by developers and 78.0% of the resolved PTD was resolved by developers within a week. We conjectured that this is mainly because the context is reviewer-centric code review process (e.g., reviewers sometimes provide constructive recommendations to help developers resolve PTD). In addition, it usually takes developers less time to resolve documentation PTD, implying that documentation PTD may be easier to resolve compared with other types of PTD. Although the majority of the identified PTD has been resolved by developers, a significant proportion of identified PTD (19.0%) was still injected into the systems without being resolved, especially for requirement PTD (i.e., only 16.7% of requirement PTD was resolved). This finding suggests a need for project managers to establish an effective mechanism for PTD management in their projects to prevent the negative impact when PTD evolves into TD. Moreover, code refactoring is the main practice used to resolve PTD identified in code reviews.

Given the importance of PTD in development, we plan to extend this work by studying PTD in code reviews in a large dataset from different communities and projects to further understand and analyze PTD identified and resolved in code reviews, the reasons of not resolving PTD, as well as the human factors in this process.

ACKNOWLEDGEMENTS

This work is funded by NSFC with No. 62172311, Hubei Provincial Natural Science Foundation of China with No. 2021CFB577, and the Special Fund of Hubei Luojia Laboratory. Amjed Tahir is supported by a NZ SfTI National Science Challenge grant (MAUX2004).

REFERENCES

- [1] Victor R. Basili, Gianluigi Caldiera, and H. D. Rombach. 1994. The Goal Question Metric Approach. *Encyclopedia of Software Engineering* (1994), 528–532.
- [2] Olga Baysal, Oleksii Kononenko, Reid Holmes, and Michael W. Godfrey. 2013. The Influence of Non-technical Factors on Code Review. In *Proceedings of the 20th Working Conference on Reverse Engineering (WCRE)*. IEEE, 122–131.
- [3] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.
- [4] Ward Cunningham. 1992. The WyCash Portfolio Management System. *ACM SIGPLAN OOPS Messenger* 4, 2 (1992), 29–30.
- [5] Rafael Maiani de Mello, Roberto Oliveira, and Alessandro Garcia. 2017. On the Influence of Human Factors for Identifying Code Smells: A Multi-Trial Empirical Study. In *Proceedings of the 11th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 68–77.
- [6] Saulo Soares de Toledo, Antonio Martini, Agata Przybyszewska, and Dag IK Sjøberg. 2019. Architectural Technical Debt in Microservices: A Case Study in a Large Company. In *Proceedings of the 2nd IEEE/ACM International Conference on Technical Debt (TechDebt)*. IEEE, 78–87.
- [7] Martin Fowler. 2009. Technical Debt Quadrant. <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>
- [8] Martin Fowler. 2018. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional.
- [9] Sávio Freire, Niccolli Rios, Boris Gutierrez, Dario Torres, Manoel Mendonça, Clemente Izurieta, Carolyn Seaman, and Rodrigo O. Spinola. 2020. Surveying Software Practitioners on Technical Debt Payment Practices and Reasons for Not Paying off Debt Items. In *Proceedings of the 24th Evaluation and Assessment in Software Engineering (EASE)*. ACM, 210–219.
- [10] Liming Fu, Peng Liang, Zeeshan Rasheed, Zengyang Li, Amjed Tahir, and Xiaofeng Han. 2022. Replication Package for the Paper: “Understanding Potential Technical Debt and Its Resolution in Code Reviews : An Exploratory Study of the OpenStack and Qt Communities”. <https://doi.org/10.5281/zenodo.6513444>
- [11] Barney G Glaser. 1965. The Constant Comparative Method of Qualitative Analysis. *Social Problems* 12, 4 (1965), 436–445.
- [12] Kazuki Hamasaki, Raula Gaikovina Kula, Norihiro Yoshida, A. E. Camargo Cruz, Kenji Fujiwara, and Hajimu Iida. 2013. Who Does What during a Code Review? Datasets of OSS Peer Review Repositories. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*. ACM, 49–52.
- [13] Xiaofeng Han, Amjed Tahir, Peng Liang, Steve Counsell, Kelly Blincoe, Bing Li, and Yajing Luo. 2022. Code Smells Detection via Modern Code Review: A Study of the OpenStack and Qt Communities. *Empirical Software Engineering* (2022).
- [14] Xiaofeng Han, Amjed Tahir, Peng Liang, Steve Counsell, and Yajing Luo. 2021. Understanding Code Smell Detection via Code Review: A Study of the OpenStack Community. In *Proceedings of the 29th IEEE/ACM International Conference on Program Comprehension (ICPC)*. IEEE, 323–334.
- [15] Toshiaki Hirao, Shane McIntosh, Akinori Ihara, and Kenichi Matsumoto. 2022. Code Reviews with Divergent Review Scores: An Empirical Study of the OpenStack and Qt Communities. *IEEE Transactions on Software Engineering* 48, 1 (2022), 69–81.
- [16] Glenn D. Israel. 1992. *Determining Sample Size*. Fact Sheet PEOD-6. Florida Cooperative Extension Service, Institute of Food and Agricultural Sciences, University of Florida, Florida, U.S.A.
- [17] Yutaro Kashiwa, Ryoma Nishikawa, Yasutaka Kamei, Masanari Kondo, Emad Shihab, Ryosuke Sato, and Naoyasu Ubayashi. 2022. An Empirical Study on Self-Admitted Technical Debt in Modern Code Review. *Information and Software Technology* 146 (2022), 106855.
- [18] Noela J. Kipyegen and William P. K. Korir. 2013. Importance of Software Documentation. *International Journal of Computer Science Issues (IJCSI)* 10, 5 (2013), 223–228.
- [19] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. 2012. Technical Debt: From Metaphor to Theory and Practice. *IEEE Software* 29, 6 (2012), 18–21.
- [20] Jason Lefever, Yuanfang Cai, Humberto Cervantes, Rick Kazman, and Hongzhou Fang. 2021. On the Lack of Consensus Among Technical Debt Detection Tools. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 121–130.
- [21] Yikun Li, Mohamed Soliman, and Paris Avgeriou. 2020. Identification and Remediation of Self-Admitted Technical Debt in Issue Trackers. In *Proceedings of the 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 495–503.
- [22] Zengyang Li, Paris Avgeriou, and Peng Liang. 2015. A systematic mapping study on technical debt and its management. *Journal of Systems and Software* 101 (2015), 193–220.
- [23] Zengyang Li, Qinyi Yu, Peng Liang, Ran Mo, and Chen Yang. 2020. Interest of Defect Technical Debt: An Exploratory Study on Apache Projects. In *Proceedings of the 36th IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 629–639.
- [24] Jiakun Liu, Qiao Huang, Xin Xia, Emad Shihab, David Lo, and Shanping Li. 2020. Is Using Deep Learning Frameworks Free? Characterizing Technical Debt in Deep Learning Frameworks. In *Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. ACM, 1–10.
- [25] Everton da S. Maldonado and Emad Shihab. 2015. Detecting and Quantifying Different Types of Self-Admitted Technical Debt. In *Proceedings of the 7th IEEE International Workshop on Managing Technical Debt (MTD)*. IEEE, 9–15.
- [26] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E Hassan. 2016. An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering* 21, 5 (2016), 2146–2189.
- [27] Aziz Nanthamornphong and Apatt Chaisutanon. 2016. Empirical Evaluation of Code Smells in Open Source Projects: Preliminary Results. In *Proceedings of the 1st International Workshop on Software Refactoring (IWor)*. ACM, 5–8.
- [28] Boris Pérez, Camilo Castellanos, Dario Correal, Niccolli Rios, Sávio Freire, Rodrigo Spinola, and Carolyn Seaman. 2020. What Are the Practices Used by Software Practitioners on Technical Debt Payment: Results from an International Family of Surveys. In *Proceedings of the 3rd International Conference on Technical Debt (TechDebt)*. ACM, 103–112.
- [29] Aniket Poddar and Emad Shihab. 2014. An Exploratory Study on Self-Admitted Technical Debt. In *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 91–100.
- [30] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164.
- [31] Klaus Schmid. 2013. On the Limits of the Technical Debt Metaphor Some Guidance on Going Beyond. In *Proceedings of the 4th International Workshop on Managing Technical Debt (MTD)*. IEEE, 63–66.
- [32] Marcelino Campos Oliveira Silva, Marco Tulio Valente, and Ricardo Terra. 2016. Does Technical Debt Lead to the Rejection of Pull Requests?. In *Proceedings of the 12th Brazilian Symposium on Information Systems (SBIS)*. ACM, 248–254.
- [33] Jie Tan, Daniel Feitosa, and Paris Avgeriou. 2021. Do Practitioners Intentionally Self-Fix Technical Debt and Why?. In *Proceedings of the 37th IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 251–262.
- [34] Dong Wang, Tao Xiao, Patanamorn Thongtanunam, Raula Gaikovina Kula, and Kenichi Matsumoto. 2021. Understanding Shared Links and Their Intentions to Meet Information Needs in Modern Code Review. *Empirical Software Engineering* 26, 5 (2021), 1–32.
- [35] Karl Eugene Wiegers. 2002. *Peer Reviews in Software: A Practical Guide*. Addison-Wesley Boston.
- [36] Farida El Zanaty, Toshiaki Hirao, Shane McIntosh, Akinori Ihara, and Kenichi Matsumoto. 2018. An Empirical Study of Design Discussions in Code Review. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, Article 11.
- [37] Nico Zazworka, Michele A. Shaw, Forrest Shull, and Carolyn Seaman. 2011. Investigating the Impact of Design Debt on Software Quality. In *Proceedings of the 2nd Workshop on Managing Technical Debt (MTD)*. ACM, 17–23.
- [38] Nico Zazworka, Rodrigo O Spinola, Antonio Vetro, Forrest Shull, and Carolyn Seaman. 2013. A Case Study on Effectively Identifying Technical Debt. In *Proceedings of the 17th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 42–47.